

SL4J LOGGING EXERCISES

EXERCISE 1: Logging Error Messages and Warning Levels

Logback.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>

  <!--
    Logback Configuration
    Controls log format, output destination, and minimum log level.
    Place this file in src/main/resources/
  -->

  <!-- Console Appender: prints logs to terminal -->
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <!-- Format: [timestamp] [LEVEL] [ClassName] - message -->
      <pattern>%d{yyyy-MM-dd HH:mm:ss} [%-5level] [%logger{36}] -
%msg%n</pattern>
    </encoder>
  </appender>

  <!-- File Appender: saves logs to a file -->
  <appender name="FILE" class="ch.qos.logback.core.FileAppender">
    <file>logs/application.log</file>
    <encoder>
      <pattern>%d{yyyy-MM-dd HH:mm:ss} [%-5level] [%logger{36}] -
%msg%n</pattern>
    </encoder>
  </appender>

  <!-- Root logger: minimum level DEBUG, outputs to both console and file -->
  <root level="DEBUG">
    <appender-ref ref="CONSOLE"/>
    <appender-ref ref="FILE"/>
  </root>

</configuration>
```

LoggingExample.java

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```

```

/**
 * LoggingExample — Demonstrates SLF4J logging levels with Logback.
 *
 * Logging Levels (LOW → HIGH severity):
 * TRACE → DEBUG → INFO → WARN → ERROR
 *
 * SLF4J acts as a facade — Logback is the actual implementation.
 * This separation allows switching logging frameworks without
 * changing application code.
 */
public class LoggingExample {

    // Logger instance — tied to this class for traceability
    private static final Logger logger =
        LoggerFactory.getLogger(LoggingExample.class);

    public static void main(String[] args) {

        logger.info("=====");
        logger.info(" SLF4J Logging Demo — All Levels ");

        logger.info("=====");

        // — TRACE: Finest detail — method entry/exit tracking —
        logger.trace("TRACE: Entering main() — finest level logging");

        // — DEBUG: Useful during development for variable inspection —
        logger.debug("DEBUG: Application started, initializing components...");
        logger.debug("DEBUG: Config loaded — log level set to DEBUG");

        // — INFO: General application lifecycle events —
        logger.info("INFO: Application is running successfully");
        logger.info("INFO: User session started");

        // — WARN: Something unexpected but recoverable —
        logger.warn("WARN: Response time exceeded threshold — 3200ms");
        logger.warn("WARN: Deprecated API method called — update recommended");

        // — ERROR: Serious failure — needs immediate attention —
        logger.error("ERROR: Database connection failed — retrying...");
        logger.error("ERROR: Payment gateway timeout — transaction rolled back");

        // — Parameterized logging — avoids string concatenation cost —
        String username = "ShreeRithi";
        int attempts = 3;
        logger.warn("WARN: User '{}' has {} failed login attempts",
            username, attempts);
    }
}

```

```

// — Logging with exception stack trace —
try {
    int result = 100 / 0; // Simulated exception
} catch (ArithmeticException e) {
    logger.error("ERROR: Arithmetic failure in calculation — {}",
        e.getMessage(), e);
}

logger.info("=====");
    logger.info("  Logging Demo Complete          ");

logger.info("=====");
}
}

```

LoggingExampleTest.java

```

import org.junit.Test;
import org.junit.Before;
import org.junit.After;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import static org.junit.Assert.*;

/**
 * LoggingExampleTest — Validates logging behaviour indirectly.
 *
 * Note: SLF4J logs are output side-effects — we test the
 * business logic that triggers the logs, not the logs themselves.
 * This is the correct and professional approach.
 */
public class LoggingExampleTest {

    private static final Logger logger =
        LoggerFactory.getLogger(LoggingExampleTest.class);

    @Before
    public void setUp() {
        logger.info("— setUp(): Test starting —");
    }

    @After
    public void tearDown() {
        logger.info("— tearDown(): Test complete —");
    }
}

```

```
//
```

```
// TEST 1: Logger instance is correctly initialized
```

```
//
```

```
@Test
```

```
public void testLoggerIsNotNull() {
```

```
    // ARRANGE
```

```
    Logger testLogger = LoggerFactory.getLogger(LoggingExample.class);
```

```
    // ACT + ASSERT
```

```
    assertNotNull("Logger instance must not be null", testLogger);
```

```
    logger.info("INFO: Logger instance validated successfully");
```

```
}
```

```
//
```

```
// TEST 2: Logger name matches class name
```

```
//
```

```
@Test
```

```
public void testLoggerName() {
```

```
    // ARRANGE
```

```
    Logger testLogger = LoggerFactory.getLogger(LoggingExample.class);
```

```
    // ACT
```

```
    String loggerName = testLogger.getName();
```

```
    // ASSERT
```

```
    assertEquals("Logger name must match class name",
```

```
        "LoggingExample", loggerName);
```

```
    logger.debug("DEBUG: Logger name verified — {}", loggerName);
```

```
}
```

```
//
```

```
// TEST 3: WARN level is enabled (logback config)
```

```
//
```

```
@Test
```

```
public void testWarnLevelIsEnabled() {
```

```
    // ARRANGE
```

```
    Logger testLogger = LoggerFactory.getLogger(LoggingExample.class);
```

```
    // ACT + ASSERT
```

```
    assertTrue("WARN level must be enabled", testLogger.isWarnEnabled());
```

```
    logger.warn("WARN: Warning level check passed");
}
```

```
//
```

```
// TEST 4: ERROR level is enabled
```

```
//
```

```
@Test
public void testErrorLevelIsEnabled() {

    // ARRANGE
    Logger testLogger = LoggerFactory.getLogger(LoggingExample.class);

    // ACT + ASSERT
    assertTrue("ERROR level must be enabled", testLogger.isErrorEnabled());
    logger.error("ERROR: Error level check passed");
}
```

```
//
```

```
// TEST 5: INFO level is enabled
```

```
//
```

```
@Test
public void testInfoLevelIsEnabled() {

    // ARRANGE
    Logger testLogger = LoggerFactory.getLogger(LoggingExample.class);

    // ACT + ASSERT
    assertTrue("INFO level must be enabled", testLogger.isInfoEnabled());
    logger.info("INFO: Info level check passed");
}
```

```
//
```

```
// TEST 6: Parameterized log message doesn't throw
```

```
//
```

```
@Test
public void testParameterizedLoggingDoesNotThrow() {

    // ARRANGE
    String user = "ShreeRithi";
    int attempt = 3;

    // ACT + ASSERT — no exception must be thrown
    try {
        logger.warn("WARN: User '{}' failed {} times", user, attempt);
    }
```

```

        assertTrue("Parameterized logging must execute without error", true);
    } catch (Exception e) {
        fail("Logging must never throw an exception: " + e.getMessage());
    }
}
}
}

```

OUTPUT :

```

Run LoggingExampleTest
6 tests passed 6 tests total, 718 ms
testParameterized 704 ms
testLoggerName 5 ms
testWarnLevel 3 ms
testInfoLevel 3 ms
testLoggerIsNot 2 ms
testErrorLevel 1 ms

"C:\Program Files\Java\jdk-25\bin\java.exe" ...
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - -- setUp(): Test starting --
2026-06-23 09:53:10 [WARN ] [LoggingExampleTest] - WARN: User 'ShreeRithi' failed 3 times
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - -- tearDown(): Test complete --
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - -- setUp(): Test starting --
2026-06-23 09:53:10 [DEBUG] [LoggingExampleTest] - DEBUG: Logger name verified - LoggingExample
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - -- tearDown(): Test complete --
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - -- setUp(): Test starting --
2026-06-23 09:53:10 [WARN ] [LoggingExampleTest] - WARN: Warning level check passed
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - -- tearDown(): Test complete --
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - -- setUp(): Test starting --
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - INFO: Info level check passed
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - -- tearDown(): Test complete --
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - -- setUp(): Test starting --
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - INFO: Logger instance validated successfully
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - -- tearDown(): Test complete --
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - -- setUp(): Test starting --
2026-06-23 09:53:10 [ERROR] [LoggingExampleTest] - ERROR: Error level check passed
2026-06-23 09:53:10 [INFO ] [LoggingExampleTest] - -- tearDown(): Test complete --

Process finished with exit code 0

```